

ONEDRAFT

CAD-AI — Aria Powered

Validation & Verification Engine

Version 1.0

Status: Implementation Specification

Database: PostgreSQL

API: REST / JSON

API Version: /api/v1

Identifiers: UUID

Primary Model: Versioned OneDraft Project Model

AI Layer: Aria Powered

Upstream Engine: Compliance & Regulatory Engine

Downstream Engine: Documentation / CAD Sheet Engine

1. VALIDATION ENGINE PURPOSE

The OneDraft Validation Engine is the deterministic verification layer between the computational project model and the generated architectural documentation.

Its purpose is to determine whether a specific version of the project model satisfies:

project requirements

geometric constraints

spatial relationships

architectural coordination

accessibility requirements

fire and life-safety requirements

building-code requirements

documentation requirements

model integrity requirements

drawing coordination requirements

The Validation Engine does not create the project model.

It does not replace the Compliance & Regulatory Engine.

It does not independently establish regulatory authority.

It evaluates the current project state against defined rules, constraints, requirements, and validation procedures.

2. VALIDATION PRINCIPLE

The fundamental validation relationship is:

```
PROJECT
↓
MODEL VERSION
↓
REVISION
↓
REQUIREMENTS
↓
CONSTRAINTS
↓
CODE MANIFEST
↓
VALIDATION RULES
↓
VALIDATION RUN
↓
VALIDATION RESULTS
↓
PASS / FAIL / WARNING / REVIEW
```

Every validation result must be traceable to the project state against which it was generated.

3. VALIDATION ENGINE DESIGN PRINCIPLE

The Validation Engine shall distinguish between:

deterministic validation

A condition that can be evaluated computationally.

rule-based validation

A condition derived from a regulatory or project rule.

geometric validation

A condition requiring geometric computation.

coordination validation

A condition requiring comparison between multiple project objects.

document validation

A condition concerning generated documentation.

human review

A condition requiring professional or customer judgment.

The engine must never represent an unresolved human-review condition as an automatically verified condition.

4. VALIDATION STATES

Validation results shall use the following primary states:

PASS
FAIL
WARNING
REVIEW_REQUIRED
NOT_APPLICABLE
BLOCKED
ERROR

Meaning:

PASS
Rule evaluated and requirement satisfied.

FAIL
Rule evaluated and requirement not satisfied.

WARNING
Condition identified but does not necessarily prevent progression.

REVIEW_REQUIRED
Automated verification is insufficient and human review is required.

NOT_APPLICABLE
Rule does not apply to the current project condition.

BLOCKED
Rule could not be evaluated because a prerequisite is missing.

ERROR
Validation execution failed.

5. VALIDATION SEVERITY

Each result shall contain a severity:

INFO
LOW
MEDIUM
HIGH
CRITICAL

Severity is independent of status.

For example:

```
status = FAIL  
severity = CRITICAL
```

or:

```
status = WARNING  
severity = LOW
```

6. VALIDATION CATEGORIES

The initial engine shall support:

CODE
SPATIAL
GEOMETRIC
STRUCTURAL_COORDINATION
FIRE_LIFE_SAFETY
ACCESSIBILITY
EGRESS
SITE
ENERGY
ASSEMBLY
MODEL_INTEGRITY
DOCUMENT
DRAWING_COORDINATION
REQUIREMENT
PROJECT_CONSTRAINT

Additional categories may be introduced without changing the core validation architecture.

7. VALIDATION RULE

The fundamental executable object is:

`validation.validation_rules`

Fields:

```
id UUID PRIMARY KEY
rule_code VARCHAR(128) NOT NULL
name VARCHAR(255) NOT NULL
description TEXT
category VARCHAR(64) NOT NULL
severity VARCHAR(32) NOT NULL
rule_type VARCHAR(64) NOT NULL
source_type VARCHAR(64)
source_reference JSONB
execution_definition JSONB NOT NULL
input_requirements JSONB
output_definition JSONB
version INTEGER NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(rule_code, version)
```

8. RULE TYPES

Initial rule types:

```
NUMERIC_THRESHOLD
BOOLEAN
ENUMERATION
RELATIONSHIP
GEOMETRIC_DISTANCE
GEOMETRIC_CONTAINMENT
GEOMETRIC_INTERSECTION
GEOMETRIC_CLEARANCE
COUNT
AREA
VOLUME
HEIGHT
RATIO
AGGREGATION
DEPENDENCY
COMPLETENESS
CONSISTENCY
CROSS_REFERENCE
DOCUMENT_EXISTENCE
DOCUMENT_COORDINATION
HUMAN_REVIEW
```

9. RULE EXECUTION MODEL

A validation rule shall conceptually execute as:

```
INPUTS
↓
OBJECT RESOLUTION
↓
PARAMETER RESOLUTION
↓
RULE EVALUATION
↓
RESULT NORMALIZATION
↓
SEVERITY ASSIGNMENT
↓
RESULT PERSISTENCE
```

The rule engine must produce a structured result rather than only a textual message.

10. VALIDATION RUN

The existing persistence model shall use:

`validation.validation_runs`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
code_manifest_id UUID
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
status VARCHAR(32) NOT NULL
result_count INTEGER DEFAULT 0
summary JSONB
```

The validation run is immutable after completion except for system-maintained execution metadata.

11. VALIDATION RUN INPUT

Every validation run shall record the exact computational inputs.

Conceptually:

```
project_id
revision_id
model_version_id
```

code_manifest_id
rule_set
validation_categories
engine_version

The engine version shall be recorded so that historical validation can be reproduced.

12. VALIDATION RESULT

The existing persistence model shall use:

validation.validation_results

Fields:

id UUID PRIMARY KEY
validation_run_id UUID NOT NULL
project_id UUID NOT NULL
revision_id UUID NOT NULL
rule_id UUID
affected_objects JSONB
category VARCHAR(64) NOT NULL
status VARCHAR(32) NOT NULL
severity VARCHAR(32) NOT NULL
description TEXT NOT NULL
source JSONB
created_at TIMESTAMPTZ NOT NULL

Additional implementation metadata may include:

observed_value
required_value
unit
tolerance
evidence
resolution

13. VALIDATION EVIDENCE

Every deterministic result should contain sufficient evidence to explain the evaluation.

Example:

```
{  
  "observed_value": 850,  
  "required_value": 900,  
  "unit": "mm",  
  "comparison": "GREATER_THAN_OR_EQUAL",  
  "object_id": "uuid",  
  "rule_code": "DOOR_CLEAR_WIDTH"
```

}

This permits the user and downstream systems to understand why the result occurred.

14. RULE SOURCE PROVENANCE

Where a validation rule originates from the Compliance & Regulatory Engine, the result shall preserve:

```
code_source_id
code_rule_id
code_manifest_id
citation
source_version
effective_date
```

The Validation Engine must never lose the regulatory provenance of a result.

15. CODE MANIFEST DEPENDENCY

Code-based validation shall execute against a specific:

```
code_manifest_id
```

The manifest identifies the applicable regulatory source set.

Therefore:

```
Project
↓
Jurisdiction
↓
Code Manifest
↓
Code Rules
↓
Validation Rules
↓
Validation Run
```

A validation result cannot be treated as jurisdictionally reproducible without the corresponding manifest.

16. PROJECT REQUIREMENT VALIDATION

Project requirements shall be validated independently from regulatory requirements.

Example:

Requirement:
Provide three bedrooms.

Model:
Two bedrooms.

Result:
FAIL

Another example:

Requirement:
Provide minimum 2,000 sq ft.

Model:
2,145 sq ft.

Result:
PASS

Project-specific requirements therefore remain independently traceable.

17. CONSTRAINT VALIDATION

Active project constraints shall be evaluated against the resulting model state.

Examples:

setback
maximum height
maximum area
lot coverage
floor-to-floor height
minimum room dimensions
required parking
required access

A model modification that violates a preserved constraint shall produce:

FAIL

and, where configured, prevent command finalization.

18. GEOMETRIC VALIDATION

The engine shall interface with the Geometry & Parametric Model Engine.

Geometric validation may include:

distance
clearance
containment
intersection
overlap
alignment
parallelism
perpendicularity
area
volume
height
offset
boundary relationship

The Validation Engine shall consume authoritative geometry outputs rather than independently creating competing project geometry.

19. GEOMETRY VERSION CONTROL

Every geometric validation shall identify:

project_id
model_version_id
object_id
geometry_version

A validation result generated from obsolete geometry shall not become the current validation state.

20. SPATIAL VALIDATION

Spatial validation evaluates relationships between spaces and architectural objects.

Examples:

room-to-room adjacency
required circulation
minimum room dimensions
required access path
space containment
space overlap
required clearances

The engine shall evaluate spatial relationships using normalized project-model data.

21. ACCESSIBILITY VALIDATION

Accessibility validation shall evaluate applicable requirements against:

- doors
- corridors
- ramps
- stairs
- washrooms
- turning spaces
- clearances
- accessible routes
- parking
- entrances
- level changes

The applicable requirements originate from the project's jurisdictional code manifest and project requirements.

Where a condition requires professional interpretation, the result shall be:

REVIEW_REQUIRED

rather than automatically marked:

PASS

22. EGRESS VALIDATION

The engine shall support computational egress checks including:

- exit count
- exit distribution
- travel distance
- dead-end conditions
- egress path continuity
- door direction
- egress width
- occupant-load relationships

Where the governing rule cannot be safely reduced to deterministic computation, the result shall identify the condition for review.

23. FIRE AND LIFE-SAFETY VALIDATION

Initial categories include:

- fire separations
- rated assemblies

protected openings
exit systems
fire barriers
occupancy relationships
travel paths
required protection

The Validation Engine shall reference assembly properties from:

`model.assemblies`

and applicable regulatory rules from:

`compliance.code_rules`

24. ASSEMBLY VALIDATION

An assembly may be validated against:

fire rating
thermal performance
acoustic performance
thickness
layer configuration
manufacturer requirements
project specifications

Example:

Required fire rating:
2 HR

Assembly:
1 HR

Result:
FAIL

25. MODEL INTEGRITY VALIDATION

Before downstream drawing generation, the model must pass internal integrity checks.

Examples:

orphaned elements
missing levels
invalid parent references
duplicate identifiers
missing geometry
invalid geometry references

broken relationships
invalid assemblies
missing required parameters

Model integrity failures shall be treated as blocking conditions where downstream generation could produce unreliable documentation.

26. RELATIONSHIP VALIDATION

The engine shall validate the project dependency graph.

Examples:

door → host wall
window → host wall
space → level
level → building
element → project
drawing → view
view → model version
revision → model version

Broken dependencies shall produce:

FAIL

or:

BLOCKED

depending on whether the problem is a violated relationship or a missing prerequisite.

27. REQUIREMENT COVERAGE

The engine shall determine whether every mandatory project requirement has a validation state.

Each mandatory requirement should reach one of:

PASS
FAIL
REVIEW_REQUIRED
NOT_APPLICABLE

A mandatory requirement with no evaluation shall not silently disappear from the validation report.

28. VALIDATION COVERAGE

A validation run shall calculate:

```
total_rules
executed_rules
passed_rules
failed_rules
warning_rules
review_required_rules
blocked_rules
error_rules
```

The summary shall also calculate coverage:

```
validation_coverage =
executed_applicable_rules /
total_applicable_rules
```

29. VALIDATION GATES

The engine shall support validation gates.

Initial gates:

```
MODEL_GATE
GEOMETRY_GATE
COMPLIANCE_GATE
DOCUMENTATION_GATE
FINALIZATION_GATE
ACCEPTANCE_GATE
```

A gate defines the minimum validation conditions required to proceed to the next stage.

30. MODEL GATE

The Model Gate verifies:

```
model integrity
object relationships
required parameters
required geometry
project requirements
active constraints
```

The model must not proceed to authoritative documentation generation when critical model-integrity failures remain unresolved.

31. GEOMETRY GATE

The Geometry Gate verifies:

- geometry completeness
- geometry validity
- geometry-version consistency
- required spatial relationships

Geometry failures must be resolved before dependent drawings are generated.

32. COMPLIANCE GATE

The Compliance Gate evaluates the results generated from the applicable code manifest.

The gate identifies:

- critical failures
- high-severity failures
- unresolved review items
- blocked rules
- missing regulatory inputs

The engine does not convert unresolved regulatory questions into automatic approval.

33. DOCUMENTATION GATE

The Documentation Gate verifies that the model contains sufficient information to generate the required drawing set.

Examples:

- required views exist
- required levels exist
- drawing references resolve
- sheet templates exist
- annotations resolve
- dimensions resolve
- schedules resolve
- title-block metadata exists

34. FINALIZATION GATE

The Finalization Gate verifies the complete provenance chain:

PROJECT
↓
MODEL VERSION
↓
REVISION
↓
CODE MANIFEST
↓
VALIDATION RUN
↓
DRAWING SET
↓
DOCUMENT PACKAGE

No finalized package may omit a required link in this chain.

35. ACCEPTANCE GATE

Customer acceptance occurs only after the required validation and documentation states have been recorded.

The acceptance process shall identify:

project_id
revision_id
model_version_id
validation_run_id
document_package_id
code_manifest_id
customer_id

This preserves the exact state accepted by the customer.

36. VALIDATION BLOCKING POLICY

The engine shall distinguish:

BLOCKING
NON_BLOCKING
REVIEW
INFORMATIONAL

A critical model integrity failure may be:

BLOCKING

A minor documentation warning may be:

NON_BLOCKING

A professional judgment condition may be:

REVIEW

37. VALIDATION PROFILE

Projects shall be able to select validation profiles.

Example:

```
RESIDENTIAL_BASIC  
RESIDENTIAL_FULL  
COMMERCIAL_BASIC  
COMMERCIAL_FULL  
PERMIT_PACKAGE  
INTERNAL_DESIGN_REVIEW  
CUSTOM
```

A validation profile determines which rule categories are executed.

38. VALIDATION PROFILE DATA

Conceptually:

`validation.validation_profiles`

Fields:

```
id UUID PRIMARY KEY  
name VARCHAR(255) NOT NULL  
description TEXT  
project_type VARCHAR(64)  
categories JSONB NOT NULL  
rule_ids JSONB  
status VARCHAR(32) NOT NULL  
created_at TIMESTAMPTZ NOT NULL  
updated_at TIMESTAMPTZ NOT NULL
```

39. RULE SET VERSIONING

Validation rules must be versioned.

Example:

```
DOOR_CLEAR_WIDTH:v1  
DOOR_CLEAR_WIDTH:v2
```

A completed validation run must retain the exact rule versions used during execution.

This prevents historical results from silently changing when rules are updated.

40. VALIDATION ENGINE VERSION

Each validation run shall record:

```
validation_engine_version  
rule_set_version  
geometry_engine_version  
model_schema_version
```

This provides computational provenance.

41. DETERMINISTIC EXECUTION

Where a validation rule is deterministic, identical inputs must produce identical results.

Conceptually:

```
same model  
+  
same code manifest  
+  
same rule versions  
+  
same engine versions  
=  
same validation result
```

This property is fundamental to reproducibility.

42. AI SEPARATION

Aria may assist with:

```
rule interpretation  
issue explanation  
resolution suggestions  
redline interpretation  
requirement classification
```

Aria shall not silently alter authoritative validation results.

The authoritative result is produced by the Validation Engine.

43. ARIA VALIDATION INTERFACE

Aria may request:

- run validation
- explain failure
- identify affected objects
- suggest correction
- compare validation runs
- summarize unresolved issues

The request passes through defined application interfaces.

Aria does not directly modify:

- validation_results
- validation_rules
- code_manifests
- audit_events

44. VALIDATION REQUEST API

Endpoint:

POST /api/v1/projects/{project_id}/validation

Request:

```
{
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "code_manifest_id": "uuid",
  "profile": "RESIDENTIAL_FULL",
  "categories": [
    "CODE",
    "GEOMETRIC",
    "ACCESSIBILITY",
    "EGRESS",
    "MODEL_INTEGRITY",
    "DOCUMENT"
  ]
}
```

Response:

```
{
  "data": {
    "validation_run_id": "uuid",
    "status": "QUEUED"
  }
}
```

```
}
```

45. VALIDATION STATUS API

Endpoint:

GET /api/v1/projects/{project_id}/validation/{validation_run_id}

Response:

```
{
  "data": {
    "validation_run_id": "uuid",
    "status": "RUNNING",
    "progress": 64,
    "results": {
      "pass": 184,
      "fail": 7,
      "warning": 11,
      "review_required": 3,
      "blocked": 0
    }
  }
}
```

46. VALIDATION RESULT API

Endpoint:

GET /api/v1/projects/{project_id}/validation/{validation_run_id}/results

Supported filters:

```
status
severity
category
rule_id
object_id
```

47. VALIDATION GATE API

Endpoint:

GET /api/v1/projects/{project_id}/validation/gates

Response:

```
{
  "data": {
    "model_gate": "PASS",
    "geometry_gate": "PASS",
    "compliance_gate": "REVIEW_REQUIRED",
    "documentation_gate": "BLOCKED",
    "finalization_gate": "BLOCKED"
  }
}
```

48. VALIDATION EXPLANATION

Every failure should be explainable through structured information.

Example:

Rule:
DOOR_CLEAR_WIDTH

Required:
900 mm

Observed:
850 mm

Difference:
-50 mm

Affected Object:
uuid

Status:
FAIL

Resolution:
Increase clear opening width to the applicable minimum.

The explanation layer may be rendered into natural language by Aria, but the underlying structured result remains authoritative.

49. VALIDATION RESOLUTION

A validation result may be associated with a corrective operation.

Conceptually:

```
validation result
↓
affected object
↓
recommended correction
```

↓
Aria command
↓
new model version
↓
new revision
↓
new validation run

The engine must not mutate historical results when a correction is made.

50. HISTORICAL VALIDATION

Every validation run remains associated with its original:

revision
model version
code manifest
rule versions
engine versions

A subsequent correction creates a new validation run.

Historical validation results remain intact.

51. VALIDATION COMPARISON

The engine shall support comparison between validation runs.

Example:

Revision 12
FAIL = 14

Revision 13
FAIL = 5

The comparison shall identify:

resolved issues
new issues
unchanged issues
regressed issues

52. REGRESSION DETECTION

A new revision shall be checked for regressions.

Example:

Revision 12:
Door D-101 = PASS

Revision 13:
Door D-101 = FAIL

The system identifies this as:

REGRESSION

Regression results shall be included in the revision validation summary.

53. AFFECTED-OBJECT ANALYSIS

When an object changes, the Validation Engine should determine which rules may be affected.

Example:

```
Wall W-201 changed
↓
Door D-201 hosted by W-201
↓
Room R-201 bounded by W-201
↓
Egress path affected
↓
Applicable validation rules queued
```

This allows incremental validation.

54. INCREMENTAL VALIDATION

The engine shall support:

FULL_VALIDATION

and:

INCREMENTAL_VALIDATION

Full validation evaluates the complete applicable rule set.

Incremental validation evaluates the affected dependency graph.

Finalization may require a full validation depending on project configuration.

55. VALIDATION DEPENDENCY GRAPH

Rules may declare dependencies:

```
rule A
  ↓
rule B
  ↓
rule C
```

Example:

```
occupant load
  ↓
required egress capacity
  ↓
required exit width
  ↓
door validation
```

A dependent rule may become:

BLOCKED

when a prerequisite rule fails or required data is unavailable.

56. VALIDATION WORKER

Expensive validation shall execute asynchronously.

Architecture:

```
API
  ↓
JOB QUEUE
  ↓
VALIDATION WORKER
  ↓
RULE EXECUTOR
  ↓
GEOMETRY ENGINE
  ↓
DATABASE
```

The worker shall update validation progress as execution proceeds.

57. VALIDATION JOB STATES

Initial states:

QUEUED
RUNNING
WAITING_FOR_GEOMETRY
VALIDATING
FINALIZING
COMPLETED
FAILED
CANCELLED

58. TRANSACTION BOUNDARY

Validation result persistence shall use transactional boundaries.

Conceptually:

BEGIN

create validation run
resolve inputs
execute rules
persist results
calculate summary
calculate gates

COMMIT

If execution fails before completion, the run shall be marked:

FAILED

rather than presenting incomplete results as a completed validation.

59. VALIDATION HASH

A completed validation run should receive a deterministic hash derived from:

project_id
revision_id
model_version_id
code_manifest_id
rule_set_version
engine_version
result_set

Conceptually:

validation_hash

This provides an integrity reference for the validation state.

60. VALIDATION PROVENANCE

The complete provenance chain becomes:

```
PROJECT
↓
REQUIREMENTS
↓
MODEL VERSION
↓
REVISION
↓
CODE MANIFEST
↓
RULE SET
↓
VALIDATION ENGINE
↓
VALIDATION RUN
↓
VALIDATION RESULTS
↓
GATES
```

This chain must remain reconstructable.

61. DOCUMENTATION HANDOFF

The Validation Engine exposes a controlled handoff to the Documentation Engine.

The Documentation Engine receives:

```
project_id
revision_id
model_version_id
validation_run_id
validation_gate_status
```

The drawing system can therefore determine whether the current project state is eligible for generation.

62. DRAWING GENERATION POLICY

The system shall distinguish:

DESIGN_DRAWING
COORDINATION_DRAWING
VALIDATION_DRAWING
PERMIT_DOCUMENTATION
FINAL_DOCUMENT_PACKAGE

Not every drawing operation requires the same validation gate.

Final permit-oriented documentation requires the strictest configured gate.

63. DOCUMENTATION COORDINATION VALIDATION

Before final document generation, the engine shall verify:

model ↔ view
view ↔ drawing
drawing ↔ sheet
sheet ↔ revision
schedule ↔ model
annotation ↔ object
dimension ↔ geometry

Broken documentation references shall produce validation results.

64. SCHEDULE VALIDATION

Schedules shall be validated against the project model.

Examples:

door schedule count = model door count
window schedule count = model window count
room schedule count = model room count

Discrepancies shall be reported.

65. DRAWING COORDINATION

The engine shall identify discrepancies such as:

plan shows door
schedule does not contain door

plan shows window

elevation does not reflect window

section cuts through object

section annotation references missing object

room number differs between plan and schedule

These become document coordination results.

66. SHEET COMPLETENESS

A final sheet set shall be checked for:

required sheets

sheet numbers

drawing titles

revision information

project information

scale

north indication where applicable

required notes

required schedules

required legends

67. FINAL PACKAGE VALIDATION

Before document-package finalization:

MODEL_GATE = PASS

GEOMETRY_GATE = PASS

COMPLIANCE_GATE = PASS or configured REVIEW state

DOCUMENTATION_GATE = PASS

The exact acceptance policy is project-configurable.

Critical failures shall remain blocking unless explicitly resolved through the appropriate project workflow.

68. HUMAN REVIEW BOUNDARY

The Validation Engine shall identify conditions requiring professional review.

Examples include:

ambiguous code applicability

complex fire strategy

alternative compliance methods
unusual occupancy conditions
jurisdiction-specific interpretation
structural engineering determination
professional certification
site-specific conditions requiring field verification

Such conditions receive:

REVIEW_REQUIRED

They are not automatically converted to:

PASS

69. PROFESSIONAL REVIEW INTEGRATION

The existing:

revisions.professional_reviews

table becomes the persistence boundary for professional review.

The relationship is:

VALIDATION RESULT
↓
REVIEW REQUIRED
↓
PROFESSIONAL REVIEW
↓
REVIEW RESULT
↓
GATE RECALCULATION

70. CUSTOMER REVIEW INTEGRATION

Customer-facing review is separate from regulatory verification.

The customer may review:

design
drawings
redlines
documentation
project requirements

Customer acceptance does not automatically convert:

FAIL

into:

PASS

The validation state remains authoritative.

71. AUDIT EVENTS

Validation activity shall create audit events.

Examples:

VALIDATION_STARTED
VALIDATION_COMPLETED
VALIDATION_FAILED
RULE_EXECUTED
VALIDATION_RESULT_CREATED
VALIDATION_GATE_CHANGED
REGRESSION_DETECTED
REVIEW_REQUESTED
REVIEW_COMPLETED

These are recorded through:

`audit.project_events`

72. SECURITY BOUNDARY

External users cannot directly modify:

`validation_results`
`validation_runs`
`validation_rules`
`code_manifests`
`validation_hashes`

All modifications occur through controlled application services.

73. API AUTHORIZATION

Validation endpoints require:

validation:read
validation:execute

Administrative rule-management operations require an elevated system role.

74. IDEMPOTENCY

Validation requests shall support:

Idempotency-Key

A repeated request with the same project state and idempotency key shall not create duplicate validation jobs.

75. OBSERVABILITY

The Validation Engine shall emit:

execution duration
rules executed
rules failed
rules blocked
geometry calls
database operations
worker utilization
error counts

Metrics shall be associated with:

project_id
validation_run_id
engine_version

where appropriate.

76. PERFORMANCE STRATEGY

The engine shall prioritize:

incremental validation
parallel rule execution
cached rule resolution
cached geometry queries
dependency-aware execution
batch database operations

Rules must not perform unnecessary repeated database queries.

77. FAILURE ISOLATION

A failure in one independent validation rule should not automatically terminate the entire validation run.

For example:

Rule A = PASS
Rule B = FAIL
Rule C = ERROR
Rule D = PASS

The run may still complete as:

COMPLETED_WITH_ERRORS

provided the engine records the failed execution explicitly.

78. ENGINE ERROR

An engine execution error differs from a project failure.

Example:

Project condition:
Door width = 850 mm
Required = 900 mm

Result:
FAIL

versus:

Rule executor:
Database unavailable

Result:
ERROR

These must never be conflated.

79. VALIDATION REPORT

The engine shall produce a structured validation report containing:

project information
revision
model version
code manifest
validation profile
engine version
rule set version
summary
gate status
critical failures
high-severity failures
warnings
review items
resolved regressions
unresolved regressions

The report may later be rendered into documentation.

80. MACHINE-READABLE VALIDATION REPORT

The canonical report format shall be JSON.

Example:

```
{
  "project_id": "uuid",
  "revision_id": "uuid",
  "model_version_id": "uuid",
  "code_manifest_id": "uuid",
  "validation_run_id": "uuid",
  "status": "COMPLETED",
  "summary": {
    "pass": 184,
    "fail": 7,
    "warning": 11,
    "review_required": 3,
    "blocked": 0
  },
  "gates": {
    "model": "PASS",
    "geometry": "PASS",
    "compliance": "REVIEW_REQUIRED",
    "documentation": "PASS"
  }
}
```

81. VALIDATION DATABASE MIGRATION

The implementation migration sequence shall extend the existing database migration architecture.

Initial validation migrations:

```
026_validation_rules
027_validation_profiles
028_validation_rule_versions
029_validation_runs
030_validation_results
031_validation_dependencies
032_validation_gates
033_validation_indexes
034_validation_seed_data
```

Migration numbers shall continue from the established OneDraft persistence layer.

82. VALIDATION INDEX STRATEGY

Primary indexes:

```
validation_run_id
project_id
revision_id
model_version_id
rule_id
status
severity
category
created_at
```

Additional indexes shall be added only after query profiling identifies the need.

83. VALIDATION SEED RULES

Development seed data shall contain representative rules such as:

```
MODEL_ELEMENT_REFERENCE
SPACE_HAS_LEVEL
ELEMENT_HAS_GEOMETRY
DOOR_CLEAR_WIDTH
ROOM_MINIMUM_AREA
ROOM_MINIMUM_DIMENSION
WALL_ASSEMBLY_PRESENT
EGRESS_PATH_CONTINUOUS
DRAWING_VIEW_VALID
SCHEDULE_MODEL_COUNT
SHEET_COMPLETENESS
```

These are development rules and shall be clearly marked as such.

84. FIRST VALIDATION INTEGRATION TEST

The first complete Validation Engine integration test shall execute:

1. Create project
2. Create requirements
3. Create building
4. Create levels
5. Create spaces
6. Create walls
7. Create doors
8. Create windows
9. Generate geometry
10. Create code manifest
11. Load validation rules
12. Start validation run
13. Execute model integrity rules
14. Execute geometric rules
15. Execute spatial rules
16. Execute accessibility rules
17. Execute egress rules
18. Execute code rules
19. Execute documentation coordination rules
20. Calculate validation summary
21. Calculate validation gates
22. Persist validation results
23. Generate validation hash
24. Create audit events

25. Retrieve validation report
26. Modify model
27. Create Revision 2
28. Run incremental validation
29. Detect regression
30. Resolve failure
31. Run full validation
32. Confirm final gate state

This becomes the first major Validation Engine regression test.

85. VALIDATION ENGINE SUCCESS CRITERIA

The Validation Engine implementation is successful when it can:

Load versioned rules

Resolve applicable rules

Consume a specific model version

Consume a specific revision

Consume a specific code manifest

Execute deterministic validation

Execute geometric validation

Execute spatial validation

Execute requirement validation

Execute model-integrity validation

Record structured results

Preserve evidence

Preserve regulatory provenance

Calculate validation gates

Detect regressions

Support incremental validation

Support full validation

Identify human-review conditions

Produce a reproducible validation report

Maintain historical validation state

86. API SUCCESS CRITERIA

A client must be able to execute:

```
CREATE PROJECT
↓
CREATE MODEL
↓
CREATE CODE MANIFEST
↓
REQUEST VALIDATION
↓
RUN VALIDATION
↓
RETRIEVE RESULTS
↓
REVIEW FAILURES
↓
MODIFY MODEL
↓
CREATE REVISION
↓
RUN INCREMENTAL VALIDATION
↓
DETECT REGRESSION
↓
RUN FULL VALIDATION
↓
PASS VALIDATION GATES
↓
HAND OFF TO DOCUMENTATION ENGINE
```

without direct access to PostgreSQL.

87. VALIDATION ENGINE HANDOFF CONTRACT

The downstream Documentation / CAD Sheet Engine receives:

project_id

revision_id
model_version_id
validation_run_id
validation_summary
validation_gates

The Documentation Engine therefore never has to infer whether the model has been validated.

Validation state is explicitly supplied.

88. ARCHITECTURAL STACK POSITION

The OneDraft computational stack now becomes:

```
PERSISTENCE
↓
API
↓
DOMAIN SERVICES
↓
PROJECT MODEL
↓
GEOMETRY & PARAMETRIC ENGINE
↓
COMPLIANCE & REGULATORY ENGINE
↓
VALIDATION ENGINE
↓
DOCUMENTATION / CAD ENGINE
↓
ARIA ORCHESTRATION
↓
CUSTOMER REVIEW
↓
ACCEPTANCE
```

The Validation Engine is therefore the principal quality-control boundary between computational design and generated architectural documentation.

89. SYSTEM OF RECORD RELATIONSHIP

The Project Model remains the authoritative representation of the design.

The Compliance Engine remains the authoritative source of the applicable regulatory manifest and rule provenance.

The Validation Engine becomes the authoritative record of the computational verification performed against those inputs.

The Documentation Engine produces derived architectural documentation from the validated project state.

90. RECONSTRUCTION REQUIREMENT

Given:

Project_ID
Revision_ID
Model_Version_ID
Code_Manifest_ID
Validation_Run_ID

the system must be capable of reconstructing:

project state
applicable requirements
applicable code sources
rules executed
validation results
gate states
validation hash

This is mandatory for reproducibility.

91. NO SILENT VALIDATION

The system shall never silently assume:

missing = pass
unknown = pass
unexecuted = pass
blocked = pass
error = pass
review required = pass

Each state remains explicit.

92. NO SILENT REGULATORY CHANGE

If the code manifest changes between revisions, the new validation run shall explicitly identify the changed regulatory input.

Conceptually:

Revision 14

Code Manifest A

Revision 15
Code Manifest B

The system records:

REGULATORY_INPUT_CHANGED

and identifies the affected validation scope.

93. NO SILENT MODEL CHANGE

A validation result is tied to an immutable model version.

A later model mutation cannot retroactively change the result.

Instead:

MODEL VERSION 17
↓
VALIDATION RUN 17A

MODEL VERSION 18
↓
VALIDATION RUN 18A

Historical results remain preserved.

94. VALIDATION AS A CONTROL SYSTEM

The Validation Engine is not merely a reporting subsystem.

It acts as a control system:

MODEL
↓
VALIDATE
↓
FAIL
↓
CORRECT
↓
REBUILD
↓
REVALIDATE
↓
PASS
↓
DOCUMENT

This loop is fundamental to OneDraft's automated drafting workflow.

95. ARIA VALIDATION LOOP

Aria may operate the correction loop through controlled commands:

```
VALIDATION RESULT
↓
ARIA ANALYSIS
↓
PROPOSED CORRECTION
↓
COMMAND
↓
MODEL REVISION
↓
VALIDATION
↓
RESULT
```

Aria does not bypass the Validation Engine.

96. REDLINE VALIDATION LOOP

Redlines shall integrate directly into validation:

```
REDLINE
↓
TARGET OBJECT
↓
ARIA COMMAND
↓
MODEL CHANGE
↓
NEW REVISION
↓
VALIDATION
↓
REDLINE RESOLUTION
```

A redline shall not be marked resolved solely because an AI response claims that the change was made.

The underlying model and validation state must support the resolution.

97. DOCUMENT PROVENANCE

The final document package shall reference:

validation_run_id
validation_hash
revision_id
model_version_id
code_manifest_id

Therefore the final architectural package is traceable to the exact computational state from which it was generated.

98. FINAL ARCHITECTURAL DECISION

OneDraft shall not treat validation as a final PDF checklist.

Validation shall exist as a computational subsystem operating directly against the versioned project model.

The architecture is therefore:

PROJECT MODEL
+
REGULATORY MODEL
+
VALIDATION RULES
+
GEOMETRY ENGINE
+
VALIDATION EXECUTION
+
PROVENANCE

The resulting validation state becomes a first-class project artifact.

99. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now contains:

Persistence/API Layer

Runtime & Application Orchestration

Computational Project Model

Geometry & Parametric Model Engine

Compliance & Regulatory Engine

Validation & Verification Engine

Revision and Provenance System

AI Command Boundary

The next major engineering layer is:

Documentation / CAD Sheet Engine

100. NEXT IMPLEMENTATION ARTIFACTS

The Validation Engine implementation package shall consist of:

Artifact A

`026_validation_engine.sql`

Database migration for validation rules, profiles, dependencies, runs, results, gates and indexes.

Artifact B

`validation_rules.py`

Typed validation-rule definitions and execution contracts.

Artifact C

`validation_engine.py`

Core rule resolution, execution, result generation and gate calculation service.

Artifact D

`validation_service.py`

Application-layer orchestration for full and incremental validation.

Artifact E

`validation_worker.py`

Asynchronous validation job processor.

Artifact F

`test_validation_engine.py`

Initial deterministic and integration test suite.

Artifact G

`openapi.yaml`

Extension of the existing API contract with validation endpoints.

101. FINAL VALIDATION ENGINE STATE

ONEDRAFT VALIDATION & VERIFICATION ENGINE v0.1

STATUS: ESTABLISHED

The Validation Engine now forms the computational quality-control boundary between the OneDraft Project Model, Geometry Engine, and Compliance & Regulatory Engine and the downstream Documentation / CAD Sheet Engine.

The next stack stage is:

DOCUMENTATION & CAD SHEET ENGINE

That layer will transform the validated computational project model into coordinated architectural views, plans, elevations, sections, details, schedules, annotations, ARCH D sheets, drawing sets, and ultimately the reproducible document package.

The architectural sequence therefore advances:

```
PROJECT MODEL
↓
GEOMETRY
↓
COMPLIANCE
↓
VALIDATION
↓
DOCUMENTATION / CAD
↓
ARIA ORCHESTRATION
↓
REDLINE ITERATION
↓
FINAL DOCUMENT PACKAGE
↓
CUSTOMER ACCEPTANCE
```

.